ARC (Agent Ready Codebase)

Laurence Chen / 陳家宏

- Clojure, dbt-Taipei 線下活動主辦人
- IT 顧問、講者、作家
- 從試算表到資料平台－重構資料工程的技術與團隊

Outline

- Agent's challenges
- Definition of ARC
- Agent Ready Codebase Principles
- Limitations & Open Questions

Agent's challenges

- Codebase 規模擴大時，能力系統性退化
- 忽略依賴
- 忽略 side effects
- 誤解 Business Logic
- Copy/Paste 而非整合，技術債累積速度超過人類
- 邊界模糊時，決策無法保持一致

ARC 設計的核心目標

- ❌ Agent 需要讀完實作、執行程式，才能理解 codebase
- ✅ 讓「程式碼結構」本身就能回答問題

Strict Doc String/Naming

- Strict Doc String
  - Every namespace must have a docstring that describes its single responsibility.
  - If new functions were added that fall outside the stated responsibility → update the docstring or extract those functions.
- Naming: Function name should describe purpose.

Data Spec - 1

- Every map that **crosses a namespace boundary** must have a named schema defined in `myapp.schema`
- Define all schemas in a dedicated myapp.schema namespace. Never define schemas inline at the call site.

Data Spec - 2

```
(ns myapp.schema
  "Single source of truth for all cross-namespace data shapes.")

(def User
  [:map
   [:id :uuid]
   [:email :string]
   [:role [:enum :admin :viewer]]])
```

Error Spec - 1

- Exception 是控制流程，不是資料——agent 無法推理「這個函數失敗時會回傳什麼形狀」
- Anomaly 的答案：**失敗也是資料**，用標準的 category 表達，而不僅只是 throw exception
- https://github.com/cognitect-labs/anomalies

Error Spec - 2

標準失敗類型：

Category	意義	對應情境
`:not-found`	資源不存在	DB 查無此筆
`:conflict`	狀態衝突	樂觀鎖失敗
`:fault`	系統內部錯誤	非預期的 exception
`:unavailable`	暫時無法服務	下游服務 timeout
`:forbidden`	權限不足	授權失敗

Error Spec - 3

```clojure
;; ❌ agent 無法推理失敗的形狀
(defn find-user [id]
  (or (db/get-user id)
      (throw (ex-info "not found" {:id id}))))

;; ✅ 失敗是資料，形狀可預期
(defn find-user [id]
  (or (db/get-user id)
      #:cognitect.anomalies{:category :not-found
                            :message  "user not found"
                            :id       id}))
```

Functional Programming

- Every function must be **pure by default**: given the same input, always return the same output, with no observable side effects.
- Never mutate arguments or external state inside a function. If transformation is needed, return a new value.
- Side effects (I/O, database writes, HTTP calls, logging) are only allowed at explicitly designated boundary layers.

Polymorphism - 1

何時該用 polymorphism？

> 當「多支援一種實作」不應該修改既有程式碼時

- **General module** 的特徵：
  - 描述「做什麼」，不想知道具體實作細節
  - 定義 interface ，在 interface 之上，加上通用操作邏輯
- **Extension** 的特徵：
  - 描述「某一種類型怎麼做」
  - 實作 interface ，在 interface 之下，實作具體細節

Polymorphism - 2

反面指標： General module 裡出現 `cond` / `case` 判斷「類型」

```
;; ❌ 類型判斷混入 general module
(defn send-notification [type payload]
  (cond
    (= type :email) (send-email payload)
    (= type :slack) (send-slack payload)))

;; ✅ general module 只依賴 protocol
(defprotocol Notifier
  (send! [this payload]))

(defrecord EmailNotifier []
  Notifier
  (send! [_ payload] (send-email payload)))

(defrecord SlackNotifier []
  Notifier
  (send! [_ payload] (send-slack payload)))
```

Log Design

- Side Effect Log ：Before/After side effect
- Exception Log 應該回答四個問題：
  - **誰**在操作？（user-id、request-id）
  - **在做什麼**？（event 名稱）
  - **資料長什麼樣**？（關鍵輸入值）
  - **哪個步驟壞掉**？（error reason)

Container/Presenter Separation

- Every UI component belongs to exactly one of two categories: container or presenter. Never mix the two responsibilities in a single component.
- A presenter renders UI from props only. It knows nothing about where data comes from, how it is fetched, or what happens after a user interaction beyond calling a callback it was given.
- A container owns data fetching, state management, and business logic. It knows nothing about how things look. Its only rendering job is to pass data down to a presenter.

Policy/Mechanism Separation

- 分離 Mechanism & Policy
  - 根據特定的線索，判斷出：部分程式碼的變動速度快 (policy)；部分則變動慢 (mechanism)。設計 or 重構時，將兩者分開。
  - https://github.com/humorless/clojure-dev-skill/blob/main/refactor-policy-mechanism.md
  - **unit test** 應該加在 mechanism 之上。

Microservice vs Monolith 的兩難

- 傳統 microservices 常見的痛點：當兩個服務需要同一段業務邏輯時，只能選：(1) 複製貼上、(2) 抽出共用 library (3) 抽出作為 service。
- 傳統 monolith 的問題：難以獨立部署，也因此難以測試局部變更。

第三選項

- Polylith 的答案：
  - 不要把「部署邊界」和「程式碼邊界」混在一起。component 是程式碼邊界，project 才是部署邊界，兩者獨立決定。
- 效果：
  - 讓你用 monolith 的開發體驗，同時保有 microservices 的部署彈性。

Polylith

```
workspace/
  components/
    user/           ← component，有 interface.clj
    payment/
    notification/
  bases/
    api/            ← HTTP 入口
    worker/         ← queue 消費者
  projects/
    api-service/    ← 包含 api base + user + payment
    worker-service/ ← 包含 worker base + payment + notification
    dev/            ← 本地開發用，包含全部
```

Why does it work?

Principle	讀完實作 or 執行程式才能理解的	看結構就能理解的
Strict Doc String	這個 module 是做什麼的	Doc string 直接告訴你
Data Spec	這個資料長什麼樣子	Schema 直接告訴你
Error Spec	失敗時會發生什麼	Anomaly category 直接告訴你
FP	這個函數有沒有改變外部狀態	Pure function 保證沒有
Polymorphism	要怎麼改，才能巧妙地擴充模組功能	Interface 直接告訴你擴充點
Log Design	Debug 時如何快速定位問題	Log 讓問題的落點容易推理
Policy/Mechanism	業務決策藏在哪裡	Policy 層直接告訴你
Container/Presenter	資料從哪裡來	Container 直接告訴你
Polylith	不同 service 共用的程式碼	模組邊界直接告訴你

Limitations & Open Questions - 1

ARC 的適用範圍

	小 Project	大 Project	Legacy Project
Agent	工具（MCP、Rules）就夠	ARC 有顯著價值	ARC 有顯著價值

Limitations & Open Questions - 2

把 ARC 轉成 Skill 的挑戰

- ARC 的 value 不是「教 LLM 不知道的東西」
  - LLM 幾乎都學過這些 principles ，知道 != 會做。
- Eval 不容易：principles 是結構性約束，沒有明確的 ground truth 。
- Skill 設計需要 language-specific，language agnostic 太抽象，agent 無法執行
- 目前進度：Policy/Mechanism 已有可運作的 refactor command ✅

Thoughts

- 有了 Coding Agent 之後
  - Programming language fluency → 📉 value down
  - Software architecture knowledge → 📈 value up
- 人的角色，從「寫 code」往「設計讓 agent 可以安全操作的結構」移動
  - Agent Ready Codebase

Q&A

LinkedIn	Substack